CO1 = Q1 + Q2 + Q3 = 30
CO2 = Q4 + Q5 = 20

**EAST WEST UNIVERSITY**
**Department of Computer Science and Engineering**
**B.Sc. in Computer Science and Engineering Program**
**Midterm Examination, Fall 2025 Semester**

| | |
|---|---|
| **Course:** | **CSE 110 Object Oriented Programming, Section 4** |
| **Instructor:** | **Dr. Md. Hasanul Ferdaus, Assistant Professor, CSE Department** |
| **Full Marks:** | 50 (achieved marks will be normalized out of 25 for final grading) |
| **Time:** | **60 minutes** |

**Note:** There are FIVE (5) questions; answer ALL of them. Course Outcomes (CO), Cognitive Levels (C), and Marks for each question are mentioned in the right margin.

**1.** Write a complete Java program that processes a dataset of integers. The program should do the following: [CO1, C2, C3, Marks: ~~5~~] 10

- Declare and initialize an integer array of size 6 within the `main()` method.
- Use a **for-each loop** to traverse all elements of the array.
- Count the number of even numbers in the array.
- Finally, display the total count of even numbers in the array.

NOTE: Use only the `main()` method — no need to define or create objects from another class.

**2.** Write a complete Java program for a simple employee payroll management system. The program should allow the creation of employee records and the computation of total monthly salary. [CO1, C2, C3, Marks: ~~5~~] 10

<u>Class Specification: **Employee**</u>

Define a class named **`Employee`** to represent an employee. This class should encapsulate the following attributes:

- **`id`**: An integer representing the employee ID.
- **`name`**: A string representing the employee's name.
- **`basicSalary`**: A double representing the base salary of the employee.

Implement the following functionalities within the **`Employee`** class:

- Constructor: Define a constructor to initialize **`id`**, **`name`**, and **`basicSalary`**.
- Method:
  - **`public double calculateSalary(double allowance)`**
    - Adds an allowance to the base salary and returns the total.
  - Parameters:
    - **`allowance`**: A **`double`** representing the allowance amount.

<u>Class Specification: **PayrollSystem** (Driver Class)</u>

- Create a class **`PayrollSystem`** containing the **`main()`** method.
- Create two employee objects using **`new`**.
- Invoke the **`calculateSalary()`** method with different allowance values.
- Display each employee's total salary.

3. Write a complete Java program for a small store to store product details using a parameterized constructor. [CO1, C2, C3, Marks: ~~7~~ 10

Class Specification: **Product**

Define a class named **`Product`** with the following attributes:

- **`productId`**: An integer representing the product ID.
- **`productName`**: A string representing the product name.
- **`price`**: A **double** representing the price.

Implement the following functionalities:

- Parameterized Constructor: Initialize all attributes using constructor parameters.
- Method:
  - **`public void showDetails()`** – displays the product information.

Class Specification: **Store** (Driver Class)

- Create a class **`Store`** containing the **`main()`** method.
- Create two **`Product`** objects using the parameterized constructor.
- Display product details using the **`showDetails()`** method.

4. Write a complete Java program to manage employee salaries using method overloading and passing objects. [CO2, C3, Marks: ~~7~~ 10

Class Specification: **Employee**

Define a class named **`Employee`** with the following attributes:

- **`empId`**: Integer
- **`empName`**: String
- **`salary`**: Double

Implement the following functionalities:

- A parameterized constructor to initialize the **`Employee`** object properly.
- Method Overloading:
  - **`public void incrementSalary(double amount)`** – increase salary.
  - **`public void incrementSalary(Employee e)`** – increase salary by another employee's salary.
- **`public void showDetails()`** – display employee info.

Class Specification: **Company** (Driver Class)

- Create two Employee objects.
- Increment the salary of the first employee using the second employee object.
- Display employee details.

5. Write a complete Java program to manage student records using constructor overloading, returning objects, and access control. ~~[CO2,~~ C3, Marks: ~~4~~ 10

<u>Class Specification: **Student**</u>

Define a class named **`Student`** with the following attributes:

- **`studentId`** (private int)
- **`studentName`** (private String)
- **`score`** (private double)

Implement the following functionalities:

- Overloaded Constructors:
  - **`Student(int studentId)`** – initializes **`studentName`** to "Unknown" and **`score`** to 0.
  - **`Student(int studentId, String studentName, double score)`** – initializes all attributes.
- Method: **`public Student higherScore(Student s)`** – returns a new Student object with the higher score among the two students.
- **`public void showDetails()`** – displays student information.

<u>Class Specification: **School** (Driver Class)</u>

- Create two Student objects using both constructors.
- Compare their scores and display the student with the higher score.